Symbio

Implementation Plan and Scalability Brief

AI-assisted circular-economy waste matchmaking - identification, deterministic matching, and explainable, safety-gated recommendations.

1. Current Status

Symbio is a working prototype, not a concept. Four development sprints have produced a full pipeline (AI identification -> knowledge-base grounding -> deterministic scoring -> safety gating -> explainable match output), a multi-tenant backend with accounts, roles and persistence (PostgreSQL or local file), a seeded demo network of 40 recipients and 15 sources around Chennai, and over 400 automated tests covering scoring determinism, safety rules, tenant isolation, and the full sign-up-to-analysis walkthrough. The codebase is deployed to GitHub and runs locally against FastAPI + Next.js with no paid API keys required.

2. Implementation Plan

Phase	Timeframe	Focus	Key deliverables
1. Pilot	Months 1-3	Validate the matching loop with real	Onboard 10-20 manufacturers (Chennai)
2. Validation	Months 4-6	Replace demo figures with verified data,	Substitute curated
3. Scale	Months 7-12	Extend geography and throughput	Move off public OSM rate limits onto a self-hosted geocoder/router; replace the

Each phase builds on infrastructure already in the codebase rather than requiring a rewrite: the tenant schema, migration ledger, RBAC model and safety layer are designed to extend by adding data and configuration, not by changing the engine.

3. Scalability

Technical scalability is addressed at each layer of the stack:

- **Backend:** FastAPI is stateless except for one in-process store of anonymous session analyses (capped at 200, the most recent). Moving that store to a shared cache (e.g. Redis) is the only change needed to run multiple backend workers behind a load balancer.
- **Database:** PostgreSQL (Supabase-compatible) with a portable, ledgered migration system; tenant isolation is enforced at the query layer (every statement filters by organisation id), so adding tenants does not require schema or code changes, only read-replica/scaling decisions as load grows.
- **AI identification:** runs on a small, self-hosted open-weight model (qwen2.5:3b via Ollama), keeping marginal inference cost near zero and avoiding per-call vendor API costs. Throughput scales by adding Ollama instances behind a simple router; a deterministic keyword/composition classifier keeps the product usable even if inference capacity is temporarily saturated.
- **Knowledge base:** growing from 29 to hundreds of materials is a data-entry task (one record per material), not an engineering task - the scoring engine's normalisation ceilings are derived from the knowledge base automatically, so new entries cannot silently distort existing scores.
- **Geography:** the public Nominatim/OSRM services used today are usage-policy-limited and adequate for a single-region pilot. Scaling beyond that is budgeted as a self-hosted geocoder/router behind the same interface, so no application code changes.
- **Correctness at scale:** the matching and safety logic are deterministic and covered by automated regression tests (over 400 at last count), which means the system can grow its data footprint - more materials, more recipients, more tenants - without silently drifting in behaviour.

4. Key Risks and Mitigations

Risk	Mitigation already in place / planned
Demo reference prices/emission factors are not verified figures	Labelled as demo/estimate in the UI today; Phase 2 replaces them with validated LCA and mark
AI identification could mislabel a hazardous material	The AI never sets safety level or invents a material; identification is grounded against the knowle
External geocoding/routing provider is a free public service with no SLA	Already wired to degrade gracefully (distance/transport shown as unavailable rather than guess
Single-process session store limits horizontal scaling	Already isolated behind one module (services/storage.py); swapping it for a shared cache is a sc

Source code: github.com/ak6580123/SYMBIO